

SOAL

1. Jelaskan mengapa Flutter dipilih sebagai framework untuk aplikasi Factory Attendance dibandingkan React Native atau native Android (Kotlin). Sebutkan minimal 3 keunggulan Flutter yang relevan dengan kebutuhan aplikasi ini. (Essay)
2. Apa perbedaan antara Firebase Authentication dan Firebase Firestore? Jelaskan peran masing-masing dalam konteks aplikasi Factory Attendance. (Essay)
3. Mengapa GPS validation dengan radius 500 meter diperlukan dalam aplikasi absensi ini? Jelaskan risiko keamanan yang mungkin terjadi jika fitur ini tidak diimplementasikan. (Analisis)
4. Jelaskan konsep Haversine Formula dan bagaimana formula ini digunakan untuk menghitung jarak antara lokasi user dan kantor dalam aplikasi Factory Attendance. (Essay)
5. Apa yang dimaksud dengan Geofencing dalam konteks aplikasi ini? Jelaskan perbedaan antara geofencing dan simple distance calculation (Haversine). Essay
6. Jelaskan perbedaan antara StatefulWidget dan StatelessWidget di Flutter. Berikan contoh konkret dari aplikasi Factory Attendance untuk masing-masing jenis widget. (Essay)
7. Mengapa state management (Provider/Riverpod) diperlukan dalam aplikasi Factory Attendance? Jelaskan masalah yang akan terjadi jika state management tidak digunakan (misal: hanya menggunakan setState). Analisis
8. Jelaskan cara kerja ML Kit Face Detection yang digunakan dalam aplikasi ini. Bagaimana library ini mendeteksi wajah, dan apa saja parameter yang bisa diatur (contoh: minFaceSize, trackingEnabled)? Essay
9. Berikut adalah skenario: User menekan tombol "Clock In" saat berada di luar radius kantor. Tuliskan pseudocode/logic flow lengkap dari tombol ditekan hingga muncul notifikasi error "Outside Area". Code/Pseudocode
10. Jelaskan bagaimana Firebase Cloud Messaging (FCM) bekerja untuk mengirim push notification saat waktu absensi hampir tiba (misal: 15 menit sebelum shift). Gambarkan arsitektur data flow dari server ke device user. Essay
11. Jelaskan konsep Clean Architecture dan bagaimana penerapannya dalam project Factory Attendance. Gambarkan layer-layer yang ada dan dependency rule-nya. Essay
12. Bagaimana cara mengamankan data attendance di Firebase Firestore? Jelaskan minimal 3 security rules yang harus diimplementasikan untuk mencegah user A melihat data attendance user B. (Security Rules)
13. Analisis risiko privasi yang mungkin timbul dari pengumpulan data GPS dan foto selfie karyawan. Bagaimana aplikasi Factory Attendance bisa meminimalisir risiko tersebut dari sisi teknis? (Analisis)
14. Jelaskan perbedaan antara Firebase Storage dan Firebase Firestore dalam konteks menyimpan foto selfie hasil absensi. Mengapa foto disimpan di Storage, bukan di Firestore? (Essay)
15. Anda diminta menambahkan fitur "Izin/Cuti" ke aplikasi Factory Attendance. Rancanglah: (a) UI/UX flow dari request hingga approval, (b) Data model Firestore

untuk menyimpan data izin, (c) Notifikasi yang dikirim ke admin saat ada request baru. (Analisis & Design)

JAWABAN INDIVIDU

BAGIAN 1: NIDA HIJRIA ALMANY (103012330169)

1. Flutter dipilih untuk aplikasi Factory Attendance dibandingkan React Native atau native Android (Kotlin) karena kalau pakai kotlin, berarti hanya bisa digunakan untuk pengguna android aja, sedangkan handphone yang ada sekarang kan ga hanya tersedia android, tapi ada juga perangkat ios (iphone), jadi agak repot kalau ingin menyesuaikan lagi, karna harus mengulang proses coding dari nol dengan menggunakan bahasa swift. Nah, dengan menggunakan flutter yang punya konsep single codebase, kita cukup menulis kode satu kali dengan menggunakan bahasa pemrograman dart, dan hasilnya langsung bisa di build menjadi aplikasi android sekaligus ios. jadi bisa menghemat waktu dan tenaga tim juga untuk membuat aplikasinya.

Selain itu, keunggulan kedua, flutter memiliki mesin rendering sendiri, artinya ia membuat setiap tombol, shadow,, dan teks langsung ke layar tanpa menumpang pada komponen bawaan sistem operasi perangkat. sehingga hasil desain tema yang sudah ditetapkan di awal bisa terlihat persis sama dan proporsional di semua tipe HP pengguna, ga bakal berantakan walaupun ukuran layarnya berbeda.

Keunggulan ketiga, saat kita sedang mendesain UI atau mencari bug, setiap menekan tombol save pada kode, perubahan visualnya langsung muncul di layar emulator dalam hitungan detik. Jadi kita tidak perlu menggunakan build ulang aplikasinya yang memakan waktu lama, sehingga proses development nya menjadi jauh lebih cepat.

2. Kalau kita bikin aplikasi, mengurus sistem login sama menyimpan data itu kan sebenarnya dua hal yang beda. Nah, Firebase nyediain dua layanan terpisah buat nanganin masing-masing tugas itu biar pengerjaan kita jadi jauh lebih gampang dan cepat.

Firestore itu fokusnya cuma buat ngurusin akses masuk aja. Ibaratnya, Auth ini kayak satpam di gerbang pabrik. Tugas satpam kan cuma ngecek ID Card dan mastiin orang yang dateng itu beneran karyawan atau bukan. Di aplikasi Factory Attendance ini, Auth kepakainya pas layar login. Dia yang bertugas ngecek apakah kombinasi NIK sama password yang diketik karyawan itu udah cocok atau belum. Auth juga yang ngatur sesi login, makanya aplikasi bisa punya fitur "Remember Me" biar karyawan ga harus ngetik password tiap hari. Tapi, Auth ini ga peduli dan ga nyimpen data kerjanya si karyawan. Dia murni cuma ngeluarin izin masuk aja.

Sedangkan Firebase Firestore itu beda lagi. Kalau Auth tadi satpamnya, nah Firestore ini ibarat lemari arsip dokumennya (atau database utama aplikasinya). Setelah karyawan berhasil login ngelewatin satpam tadi, semua aktivitas yang dia lakukan di dalam aplikasi bakal disimpan ke Firestore. Di proyek kita ini, Firestore dipake buat nyimpen data profil (misalnya nentuin dia itu role-nya Admin atau Employee). Terus yang paling utama, tiap kali karyawan nekan tombol clock-in atau clock-out, datanya masuk ke sini. Data yang dicatat lumayan lengkap, mulai dari waktu dia nekan tombol (timestamp), titik lokasi koordinat GPS-nya, status kehadiran, sampai link URL foto selfie-nya.

Jadi kesimpulannya Auth itu khusus nanganin "kamu siapa dan boleh masuk atau ngga", sedangkan Firestore itu khusus nyimpen "data apa aja yang kamu lakukan di dalam aplikasi setelah berhasil masuk". Dua tugas ini emang harus dipisah biar datanya aman dan struktur kodenya lebih rapi.

3. Fitur GPS validation dengan radius 500 meter ini wajib banget ada buat mastiin karyawan beneran udah sampai di area pabrik pas mereka mau absen. Konsepnya kan ini aplikasi buat ngontrol absensi industri, jadi pihak HRD harus dapet data lokasi yang valid dan terpercaya. Kalau fitur pengecekan radius ini ga dibikin, risiko keamanannya lumayan fatal buat perusahaan.

Risiko pertama, bakal gampang banget kejadian kecurangan waktu (time fraud). Karyawan bisa aja baru bangun tidur, masih rebahan di kosan atau lagi nongkrong di warung kopi, tapi udah nekan tombol clock-in di HP-nya biar dianggep dateng tepat waktu. Nanti perusahaan bisa rugi karena bayar gaji full buat orang yang aslinya telat atau malah bolos kerja.

Risiko kedua, tanpa adanya batasan radius 500 meter, data statistik absensi yang ditampilkan di layar Admin Dashboard bakal jadi ga akurat. Kan HRD butuh report buat liat tren performa karyawan setiap minggunya, nah kalau datanya dimanipulasi sama karyawan dari luar pabrik, KPI (Key Performance Indicator) aplikasinya jadi ga ada gunanya. Selain itu, angka 500 meter ini dirasa pas, soalnya kalau terlalu sempit (misal cuma 50 meter), kasian juga karyawan yang udah di gerbang pabrik tapi gagal absen cuma gara-gara sinyal GPS di HP-nya lagi agak lompat-lompat atau kurang akurat. Jadi ngasih kelonggaran sedikit tapi sistemnya tetep aman dari akal-akalan.

4. Haversine Formula itu intinya adalah sebuah rumus matematika yang dipake buat ngitung jarak terpendek antara dua titik di atas permukaan bumi. Bentuk bumi kita kan bulat (bola), bukan bidang datar biasa kayak kertas. Jadi kita ga bisa asal ngitung jarak pakai rumus garis lurus biasa karena ada lengkungan buminya. Nah, rumus Haversine ini ngegunain garis lintang (latitude) sama garis bujur (longitude) dari dua titik buat ngedapetin jarak aslinya di dunia nyata dalam hitungan meter atau kilometer.

Di dalam aplikasi Factory Attendance ini, penggunaannya cukup simpel tapi penting banget. Pas karyawan nekan tombol buat clock-in, plugin geolocator di Flutter bakal ngambil koordinat latitude sama longitude posisi HP karyawan saat itu juga. Di sisi lain, dalam kodenya kita udah nentuin titik latitude sama longitude patokan pabriknya secara statis.

Rumus Haversine ini bakal masukin kedua pasang koordinat itu (titik lokasi karyawan dan titik lokasi pabrik) buat dikalkulasi. Hasil dari hitungan rumus ini adalah angka jarak dalam satuan meter. Terus, hasil jaraknya ini bakal dicek sama logika aplikasi kita. Kalau hasilnya di bawah atau sama dengan 500 meter, sistem bakal nerima dan nampilin status "In Area" biar karyawan bisa lanjut ke tahapan foto selfie. Tapi kalau hasil hitungannya lebih besar dari 500 meter, sistem bakal otomatis nolak dan ngeluarin peringatan "Outside Area".

5. Geofencing dalam konteks aplikasi Factory Attendance adalah pembuatan batas radius virtual atau semacam pagar digital di sekitar titik lokasi spesifik. Di proyek aplikasi kita ini, batas virtual yang ditetapkan adalah radius sejauh 500 meter, dengan titik pusatnya berada tepat di koordinat latitude dan longitude kantor pabrik. Jadi, sistem akan membentuk area lingkaran di peta, dan area inilah yang menjadi zona valid untuk karyawan melakukan absensi.

Perbedaan utama antara GPS tracking biasa dengan Geofencing sebenarnya cukup mendasar. Kalau kita hanya menggunakan fitur GPS tracking biasa, sistem hanya akan membaca dan menampilkan posisi pengguna pada saat itu saja. Misalnya, aplikasi sekadar mendapatkan titik koordinat dan menampilkannya di atas peta Google Maps, tanpa ada tindak lanjut otomatis dari data tersebut. Bisa dibilang, GPS tracking sifatnya pasif karena tugasnya murni hanya melaporkan lokasi.

Sebaliknya, Geofencing sifatnya jauh lebih aktif dan memiliki logika validasi. Geofencing tidak hanya mencari tahu di mana lokasi karyawan, tetapi juga secara otomatis mengecek apakah titik lokasi karyawan tersebut berada di dalam atau di luar pagar radius 500 meter yang sudah kita buat tadi. Dalam aplikasi absensi ini, Geofencing bertugas memicu perubahan status. Ketika lokasi karyawan masuk ke dalam lingkaran batas tersebut, sistem akan mendeteksi status transisi (enter) dan mengubah indikator menjadi "In Area", sehingga tombol untuk clock-in bisa diakses. Namun kalau karyawan masih di luar batas, sistem langsung memblokirnya dan menampilkan peringatan "Outside Area". Jadi, Geofencing membuat data lokasi GPS menjadi lebih fungsional karena ada aturan khusus yang mengikatnya.

6. Di Flutter, perbedaan paling mendasar antara StatelessWidget dan StatefulWidget itu ada di kemampuannya buat ngerubah tampilan setelah pertama kali di-load di layar. StatelessWidget itu seperti poster yang statis. Sekali kita bikin dan tampilkan layarnya, ya udah gitu aja, tampilannya ga bakal berubah lagi walaupun ada interaksi dari user-nya. Widget jenis ini cocok banget dipake buat ngebangun bagian antarmuka

(UI) yang memang ga butuh perubahan data secara real-time. Kalau di aplikasi Factory Attendance yang lagi kita bangun, contoh StatelessWidget itu kaya komponen teks judul di halaman Dashboard, atau layout dasar yang cuma nerapin tema warna Deep Navy. Karena widget ini ga nyimpen data yang berubah-ubah, kerjanya jadi jauh lebih enteng dan bikin performa aplikasi tetep cepet.

Nah, kebalikannya nih, StatefulWidget itu sifatnya dinamis karna dia punya "state" internal. Artinya, widget ini bisa ngerespon interaksi pengguna atau perubahan data dari sistem, terus ngerender ulang tampilannya biar datanya selalu update (biasanya pake perintah setState). Kita wajib pake ini kalau ada elemen di layar yang tampilannya harus berubah-ubah. Contoh paling jelas di aplikasi absensi kita adalah halaman FaceCaptureScreen. Di layar itu kan kamera depan harus nyala dan nampilin preview yang gerak terus buat deteksi wajah. Belum lagi ada indikator peringatan blur atau lighting yang teksnya bisa gonta-ganti sesuai kondisi cahaya pas karyawan mau selfie. Selain itu, tombol "Clock In" yang awalnya cuma teks biasa, terus pas dipencet berubah jadi animasi loading muter-muter pas nunggu pencarian lokasi GPS, nah itu juga murni harus pake StatefulWidget. Intinya, kalau layarnya butuh update visual di tengah jalan, kita pasti butuh widget yang stateful ini.

7. Penggunaan state management seperti Provider atau Riverpod sifatnya sangat wajib di aplikasi Factory Attendance ini. Hal ini dikarenakan alur aplikasi kita lumayan kompleks. Kita harus menggabungkan proses pengambilan titik lokasi GPS secara real-time, menjalankan deteksi wajah dari kamera depan, sampai mengirim datanya ke database Firebase. Fungsi utama dari state management ini adalah untuk memisahkan bagian kode yang mengurus tampilan layar (UI) dengan bagian kode yang menangani logika bisnis aplikasinya. Dengan pemisahan ini, struktur kode yang kita tulis bakal jauh lebih rapi, gampang dibaca, dan logikanya ga menumpuk berantakan di satu file UI saja. Bakal banyak masalah yang muncul kalau kita ga pakai state management dan cuma mengandalkan fungsi bawaan seperti setState. Masalah pertama adalah prop drilling. Misalnya, pas kita mau memindahkan data status login karyawan atau data titik koordinat GPS ke halaman yang posisinya ada di dalam struktur komponen, kita harus mengoper data itu secara manual lewat parameter berkali-kali. Ini bakal bikin proses coding jadi repot banget dan rentan terkena error. Masalah kedua pastinya ngaruh banget ke performa aplikasinya. Kalau kita memanggil perintah setState di komponen UI yang ukurannya besar, sistem Flutter bakal memaksa semua elemen di bawahnya buat digambar ulang dari nol. Padahal, bisa aja yang perlu diubah warnanya cuma satu teks kecil. Proses gambar ulang yang ga perlu ini jelas bakal membuang banyak memori HP dan bikin transisi animasinya jadi kurang mulus. Dengan memakai Provider atau Riverpod, kita bisa memastikan kalau cuma bagian UI yang datanya beneran berubah aja yang bakal di-update ke layar.
8. ML Kit Face Detection yang kita pakai buat validasi selfie absensi ini memproses datanya murni di dalam perangkat HP pengguna (on-device) tanpa harus mengirim gambar tersebut ke server luar terlebih dahulu. Cara kerjanya, algoritma bawaan dari

library ini bakal memindai tiap frame gambar yang ditangkap oleh kamera depan secara real-time. Begitu mendapatkan gambar yang menyerupai wajah, sistem akan mencari letak titik-titik penting (landmark) seperti posisi mata, hidung, dan mulut buat memastikan kalau objek tersebut benar-benar wajah manusia. Di aplikasi Factory Attendance ini, ML Kit diatur buat memenuhi beberapa syarat validasi yang cukup ketat. Kita membatasi sistem supaya cuma nerima pas ada persis 1 wajah di layar, dan bakal otomatis nolak (error state) kalau kamera menangkap lebih dari satu wajah sekaligus (multiple faces). Selain itu, ada juga fitur analisis ketajaman buat mendeteksi apakah hasil kameranya blur atau ngga, serta mengecek indikator pencahayaan di sekitar karyawan. Tombol capture baru bisa ditekan kalau semua validasi ini sudah sesuai. Kita juga bisa mengatur beberapa parameter bawaan dari ML Kit biar deteksinya lebih presisi. Misalnya ada parameter minFaceSize yang bisa diatur supaya wajah user harus memenuhi persentase ukuran tertentu dari layar, yang mana ini membantu banget buat ngepasin wajah karyawan dengan alignment frame overlay. Ada juga parameter trackingEnabled yang kalau diaktifkan bakal memberikan ID unik ke wajah yang terdeteksi, jadi sistem bisa terus melacak identitas wajah yang sama walaupun HP-nya agak goyang pas karyawan lagi nyari posisi selfie yang pas.

9. Skenarionya kan begini, ada karyawan yang mau mencoba menekan tombol "Clock In" padahal posisi dia masih berada di luar area radius 500 meter dari pabrik. Logika dasar buat menangani kondisi ini sebenarnya berjalan berurutan dari saat tombol dipencet sampai akhirnya muncul peringatan "Outside Area".

Pertama, pas user menekan tombol absen, aplikasi bakal langsung menampilkan animasi loading di layar biar user tau sistemnya lagi memproses data. Kedua, aplikasi bakal memanggil fungsi dari plugin GPS (geolocator) buat mendapatkan data koordinat latitude sama longitude posisi HP karyawan saat itu juga. Ketiga, koordinat karyawan tersebut bakal dibandingkan sama titik koordinat kantor pabrik yang udah kita simpan secara statis di dalam kodenya. Keempat, sistem bakal menghitung selisih jarak di antara dua titik koordinat tersebut menggunakan rumus Haversine. Nah, di tahap kelima ini ada pengecekan kondisi (if). Kalau hasil perhitungan jaraknya ternyata lebih besar dari 500 meter, proses absensi bakal langsung dihentikan, loading UI dimatikan, dan aplikasi mengeluarkan peringatan yang isinya status "Outside Area". Proses selanjutnya buat deteksi wajah ga bakal dilanjutkan sama sekali.

Biar lebih kebayang, berikut ini rancangan pseudocode-nya:

```
function prosesAbsensi() {  
    // Tampilkan animasi loading  
    tampilkanLoadingUI();  
  
    try {  
        // Dapatkan posisi GPS user saat ini  
        Lokasi kordinatUser = getPosisiSekarang();
```

```

// Titik lokasi pabrik
Lokasi kordinatPabrik = Lokasi(lat: -6.2088, lng: 106.8456);

// Hitung jarak dalam meter
double hitungJarak = hitungPakaiHaversine(kordinatUser, kordinatPabrik);

// Validasi pengecekan radius
if (hitungJarak > 500.0) {
    // Jika di luar radius, hentikan proses
    sembunyikanLoadingUI();
    tampilkanPesanError("Outside Area: Anda belum berada di area pabrik.");
    return;
}

// Jika masih dalam radius 500m, buka kamera deteksi wajah
sembunyikanLoadingUI();
navigasiKeLayarFaceDetection();

} catch (error) {
    sembunyikanLoadingUI();
    tampilkanPesanError("Sistem gagal melacak lokasi Anda.");
}
}

```

10. Firebase Cloud Messaging (FCM) itu pada dasarnya adalah layanan yang tugasnya mengirimkan pesan atau push notification langsung ke HP pengguna secara real-time. Di dalam aplikasi absensi ini, FCM kepake banget buat ngirim notifikasi pengingat secara otomatis, misalnya buat ngingetin karyawan 15 menit sebelum jam kerja (shift) mereka dimulai.

Cara kerja dan alur pergerakan datanya cukup terstruktur. Pertama, saat karyawan baru pertama kali login ke aplikasi di HP-nya, FCM bakal nge-generate sebuah Device Token. Token ini bentuknya string acak yang berfungsi sebagai penanda unik buat mengenali perangkat spesifik milik karyawan tersebut. Aplikasi kita kemudian bakal nyimpen Device Token ini ke dalam database Firestore supaya tersambung sama data profilnya.

Selanjutnya, buat fitur pengingat jadwal, kita biasanya butuh menjalankan skrip backend atau Firebase Cloud Functions. Skrip backend ini berjalan otomatis di server buat mengecek waktu. Ketika waktunya udah masuk 15 menit sebelum shift absen dimulai, backend bakal mencari data Device Token milik karyawan yang masuk kerja hari itu.

Setelah tokennya ketemu, backend bakal merakit payload (isi pesan yang memuat judul dan isi notifikasinya) dan ngirim data tersebut ke infrastruktur server FCM milik Google. Terakhir, server Google ini yang bakal mencari perangkat Android yang cocok dengan token tadi, dan mendistribusikan notifikasinya. Sistem Android bakal menerima data itu dan langsung nampilin pop-up pengingat di layar HP karyawan, walaupun aplikasinya lagi ga dibuka.

11. Konsep Clean Architecture pada dasarnya adalah pola yang digunakan untuk menyusun folder dan struktur kode supaya aplikasi kita lebih rapi, terorganisir, dan gampang dikembangkan buat jangka panjang. Inti dari arsitektur ini adalah memisahkan kode menjadi beberapa lapisan (layer) yang punya tanggung jawab masing-masing, sehingga urusan tampilan antarmuka tidak tercampur dengan urusan pengelolaan database atau logika sistem. Di project Factory Attendance ini, penerapannya sangat membantu apalagi dikerjakan secara berkelompok, karena tiap anggota bisa fokus di bagian tertentu tanpa merusak fungsi yang lagi dikerjakan anggota lain.

Dalam penerapannya di project kita, ada tiga layer utama yang digunakan. Pertama adalah layer Presentation, ini bagian terluar yang tugasnya murni cuma menangani urusan antarmuka atau UI (seperti halaman Dashboard, komponen tombol) dan state management menggunakan Provider. Kedua, layer Domain yang posisinya ada di tengah. Bagian ini berisi aturan bisnis utama aplikasinya (seperti usecases dan entities). Bagian ini sama sekali tidak peduli dari mana datanya berasal, entah itu dari Firebase atau memori lokal. Ketiga adalah layer Data, tugasnya khusus berkomunikasi dengan pihak luar, seperti mengambil koordinat dari GPS atau membaca dokumen riwayat absensi dari Firebase Firestore, lalu mengubah formatnya agar bisa dipahami oleh layer Domain.

Hal yang membuat arsitektur ini bisa berjalan dengan baik adalah aturan ketergantungannya atau yang biasa disebut dependency rule. Aturannya sangat ketat: layer bagian dalam sama sekali tidak boleh tahu atau bergantung dengan layer yang ada di luarnya. Jadi, layer Domain ga boleh tau soal pengaturan Firebase atau bentuk UI-nya, tapi layer Presentation diperbolehkan buat memanggil fungsi yang ada di layer Domain. Dengan aturan satu arah ini, kalau suatu saat perusahaan mau ganti Firebase dengan database lain, bagian UI dan aturan bisnisnya ga perlu dirombak total dari nol.

12. Mengamankan data absensi di dalam Firebase Firestore itu wajib dilakukan lewat pengaturan Security Rules, apalagi ini menyangkut privasi pergerakan posisi karyawan tiap harinya. Secara bawaan sistem, Firestore lumayan terbuka kalau kita ngga mengatur pembatasannya dari awal. Makanya, kita harus merancang aturan yang ketat biar data absensi satu orang ga bocor dan ga bisa diintip sama karyawan lainnya.

Ada minimal tiga aturan keamanan utama yang harus kita implementasikan buat mencegah hal ini. Aturan yang pertama adalah wajib login (Authentication Check). Kita harus memblokir semua upaya untuk membaca atau menulis data kalau pengguna tersebut belum melakukan proses login secara valid menggunakan Firebase Auth. Lewat aturan ini, orang iseng dari luar yang ga punya akun atau NIK terdaftar dipastikan ga bakal bisa mengakses koleksi data apa pun.

Aturan kedua adalah isolasi data antar pengguna, ini bagian paling penting buat mencegah User A melihat data absensi milik User B. Di Firestore, kita bikin aturan yang membandingkan apakah ID pengguna (UID) yang lagi membuka aplikasi itu sama dengan ID pengguna yang tersimpan di dalam dokumen absensi tersebut. Kalau ID-nya cocok, berarti sistem ngasih izin dia buat ngebuka dan nambahin riwayat absennya sendiri. Tapi begitu dia mencoba ngebuka dokumen absensi orang lain yang ID-nya berbeda, Firebase bakal langsung otomatis menolak perintahnya karena dianggap melanggar privasi.

Aturan yang ketiga adalah memberikan akses istimewa buat peran Admin. Walaupun datanya tadi udah diisolasi per orang, pihak HRD kan tetap butuh melihat seluruh data untuk memantau performa harian di halaman Admin Dashboard. Jadi kita perlu nambahin pengecekan kondisi khusus di aturannya. Sistem bakal ngecek apakah akun yang sedang mengakses tersebut punya atribut status 'admin'. Kalau valid, barulah Firebase mengizinkan akun HRD tersebut untuk membaca seluruh koleksi data milik semua karyawan tanpa terkena batasan isolasi ID tadi.

13. Aplikasi berbasis HRIS seperti Factory Attendance ini emang lumayan rentan bersinggungan dengan masalah privasi data. Hal ini karena kita secara langsung mengumpulkan titik lokasi GPS presisi dan mengambil foto selfie wajah karyawan secara rutin setiap hari kerja. Risiko privasi pertama yang paling mengkhawatirkan adalah kemungkinan terjadinya pelacakan lokasi di luar jam kerja. Karyawan pasti bakal ngerasa sangat terganggu privasinya kalau aplikasi ini diem-diem terusantau pergerakan mereka padahal mereka lagi libur atau lagi di rumah. Risiko kedua adalah terjadinya kebocoran identitas biometrik wajah. Data foto wajah itu sifatnya sangat personal, kalau sampai tersebar luas ke publik karena penyimpanan sistem yang kurang aman, bisa aja datanya disalahgunakan oleh pihak ga bertanggung jawab buat tujuan lain.

Buat meminimalisir risiko-risiko tersebut dari segi teknis, aplikasi Factory Attendance kita udah nerapin beberapa prosedur keamanan. Khusus buat urusan GPS, kita mengatur alur aplikasinya supaya cuma mengaktifkan sensor dan mengambil koordinat lokasi tepat pada detik saat karyawan memencet tombol "Clock-in" atau "Clock-out". Sistem dilarang keras mengaktifkan layanan pemantauan lokasi secara terus-menerus di background HP. Jadi setelah karyawan selesai absen dan data lokasinya dikirim, pelacakan GPS tersebut langsung dimatikan saat itu juga.

Sedangkan untuk meminimalisir risiko kebocoran foto selfie wajah, sistem pengamanannya diterapkan di bagian database. Semua foto yang dikirim dari kamera HP bakal disimpan langsung ke dalam Firebase Storage yang dilindungi oleh Security Rules tingkat tinggi. Aturan ini memastikan bahwa tautan URL foto tersebut bersifat tertutup dan ga bisa diakses sembarangan lewat internet. Yang punya izin buat melihat dan memuat gambar tersebut cuma akun milik karyawan yang bersangkutan dan akun HRD saja. Selain itu, aplikasi juga didesain untuk tidak menyimpan salinan foto selfie absensi ke dalam galeri memori HP pengguna biar fotonya ga rawan tersebar kemana-mana.

14. Dalam pengerjaan aplikasi kita, Firebase menyediakan dua tempat yang fungsinya emang dibedakan antara Firebase Storage sama Firebase Firestore. Perbedaan ini kerasa banget waktu kita berurusan sama penyimpanan hasil foto selfie karyawan. Firebase Storage itu secara spesifik dirancang dan dimaksimalkan performanya buat menyimpan file mentah yang ukurannya cukup besar. File mentah ini contohnya kayak dokumen PDF, video, rekaman suara, atau gambar dalam format JPG dan PNG dari hasil kamera HP pengguna. Jadi segala macam bentuk file fisik yang mau kita simpan di cloud, tempatnya udah pasti di Storage.

Di sisi lain, Firebase Firestore fungsinya buat menyimpan data teks terstruktur yang bentuknya dokumen berbasis JSON. Firestore ini sangat gesit kalau ditugaskan buat mengolah pencarian data (query) yang berupa teks pendek, kombinasi angka, penanda waktu (timestamp), atau status benar/salah (boolean).

Nah, alasan kuat kenapa kita menaruh foto selfie di Storage dan bukan langsung dimasukin ke Firestore adalah karena masalah batasan kapasitas dan kecepatan pencarian data. Kalau kita memaksakan mengubah file foto menjadi kode teks yang panjang banget dan menyimpannya di Firestore, sistemnya bakal macet. Ini dikarenakan Firestore ngasih batas maksimal ukuran cuma 1 Megabyte saja untuk satu dokumen. Sementara foto tangkapan HP jaman sekarang resolusinya tajam dan ukuran filenya pasti jauh melebihi batas itu. Selain itu, data yang terlalu gemuk bakal bikin proses memuat riwayat kehadiran jadi super lambat. Alur yang paling benar dan sesuai standar teknis adalah: aplikasi mengirimkan file gambarnya ke Storage, setelah sukses Storage bakal menghasilkan tautan URL pendek. Nah, cuma teks pendek berupa tautan URL itulah yang kemudian kita simpan di dalam dokumen Firestore berdampingan sama teks data absensi lainnya.

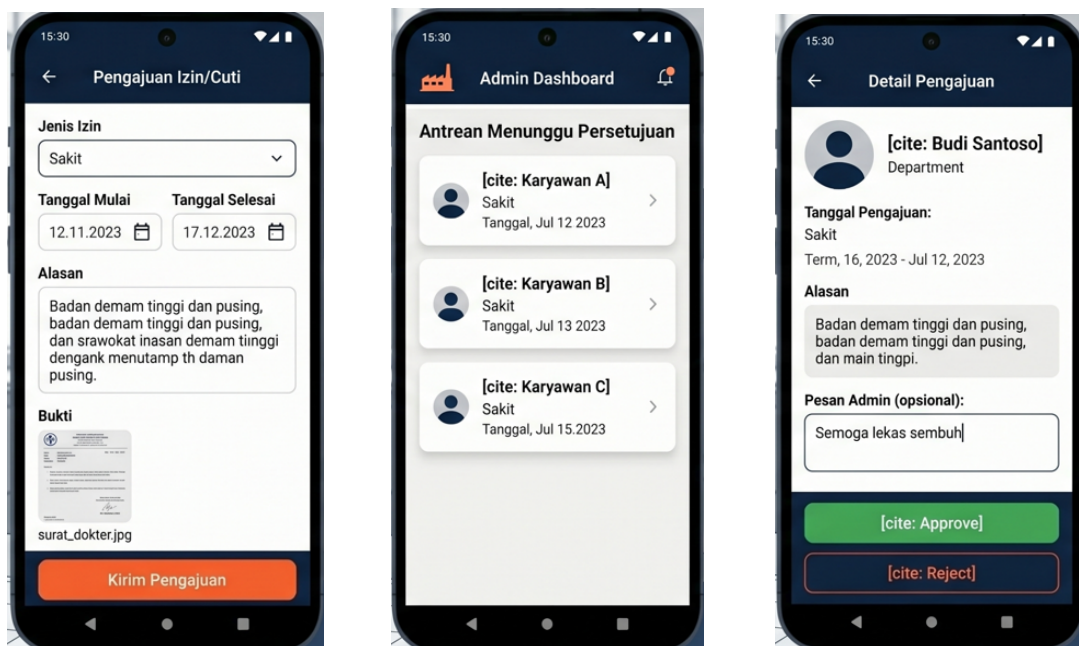
15. Buat menambahkan fitur pengajuan Izin atau Cuti ke dalam aplikasi Factory Attendance, kita perlu merancang alur lengkap mulai dari tampilan pengguna, struktur tabel datanya, sampai notifikasi otomatisnya.

- a. UI/UX Flow dari Request hingga Approval

Alur antarmuka pengguna untuk fitur izin atau cuti ini dibagi menjadi dua sisi, yaitu dari sisi karyawan yang mengajukan dan sisi admin HRD yang memberikan

persetujuan. Dari sisi karyawan, alurnya dimulai saat karyawan membuka halaman Profile dan menekan tombol baru bernama "Ajukan Izin/Cuti". Setelah itu, layar akan berpindah ke halaman form pengajuan. Di form ini, karyawan harus mengisi beberapa atribut, seperti memilih jenis izin (sakit, cuti tahunan, atau keperluan pribadi) melalui dropdown, memilih rentang waktu pada date picker untuk tanggal mulai dan selesai, mengisi kolom teks alasan secara detail, dan menekan tombol unggah untuk melampirkan foto bukti seperti surat dokter. Saat tombol submit ditekan, layar menampilkan indikator loading singkat dan memunculkan pesan berhasil. Data tersebut kemudian akan masuk ke halaman riwayat absensi karyawan dengan status berlabel "PENDING".

Untuk bagian admin, alurnya dimulai ketika admin menerima push notification di layar HP-nya. Saat notifikasi tersebut diklik, aplikasi akan langsung membuka halaman Admin Dashboard tepat di bagian "Pending Approvals". Admin bisa menekan nama karyawan yang mengajukan untuk melihat detail alasan dan mengecek foto surat bukti yang dilampirkan. Di bagian bawah layar detail tersebut, terdapat tombol "Approve" dan "Reject". Saat admin memilih salah satu tombol, akan muncul sebuah pop-up dialog untuk mengetikkan catatan tambahan, misalnya "Semoga lekas sembuh". Setelah dikonfirmasi, status pengajuan otomatis berubah di database dan data tersebut hilang dari daftar antrian persetujuan.



b. Data Model Firestore

Untuk menyimpan data pengajuan izin ini dengan aman dan terstruktur di Firestore, kita perlu membuat sebuah collection baru dengan nama "leave_requests". Setiap pengajuan baru dari karyawan akan disimpan sebagai satu dokumen terpisah di dalam koleksi tersebut dengan ID dokumen yang dibuat otomatis oleh sistem Firebase.

Di dalam setiap dokumen pengajuan, terdapat beberapa field data beserta tipe datanya. Field yang dibutuhkan antara lain: "id_karyawan" bertipe String yang berisi UID karyawan dari Firebase Auth, "nama_karyawan" bertipe String untuk kemudahan HRD membaca data tanpa query ganda, "jenis_izin" bertipe String yang menyimpan pilihan kategori izin, "tanggal_mulai" dan "tanggal_selesai" yang keduanya menggunakan tipe data Timestamp, "alasan" bertipe String untuk menyimpan paragraf keterangan lengkap, "url_bukti" bertipe String yang berisi tautan gambar dari Firebase Storage, "status" bertipe String yang nilainya secara bawaan adalah PENDING lalu bisa berubah menjadi APPROVED atau REJECTED, dan "catatan_admin" bertipe String yang di awal berupa teks kosong dan akan diisi saat proses approval.

c. Notifikasi yang Dikirim ke Admin

Agar HRD dapat langsung merespons pengajuan yang masuk tanpa harus mengecek aplikasi secara manual terus-menerus, fitur ini menggunakan Firebase Cloud Messaging (FCM) dan Firebase Cloud Functions. Skenario kerjanya berjalan secara otomatis di belakang layar. Pemicu utamanya adalah ketika fungsi Cloud Functions mendeteksi adanya kejadian penambahan dokumen baru (trigger onCreate) di dalam koleksi "leave_requests".

Begitu pengajuan masuk, skrip backend akan memfilter database profil untuk mencari akun pengguna yang memiliki peran atau role sebagai admin. Sistem kemudian mengambil data Device Token FCM dari perangkat HP para admin tersebut. Setelah token didapatkan, server akan mengirimkan instruksi push notification yang berisi judul "Pengajuan Izin Baru" dan isi pesan singkat seperti "[Nama Karyawan] baru saja mengajukan izin. Silakan ketuk untuk meninjau detail dan memberikan persetujuan". Notifikasi ini akan langsung muncul dan bergetar di layar HP admin walaupun aplikasi Factory Attendance sedang ditutup.

BAGIAN 2: FAJRIL IKHSAN RAMADHAN (103012300204)

1. Ada beberapa hal yang menjadi pertimbangan mengapa memilih Flutter untuk aplikasi Factory Attendance dibandingkan React Native atau native Kotlin:
 - a. Single Codebase: Kita cuma butuh nulis satu kode (menggunakan Dart) untuk nge-build aplikasi ke Android dan iOS sekaligus. Menghemat waktu development banget dibandingkan kalau pakai Kotlin (yang cuma jalan di Android).
 - b. Performa UI (Custom Rendering Engine): Flutter punya rendering engine sendiri (Skia/Impeller), jadi UI-nya bakal konsisten di device apa pun. Dia nggak bergantung sama komponen bawaan OS yang sering bikin layout berantakan antar versi OS.
 - c. Hot Reload: Buat tweak UI, fitur ini ngebantu banget untuk lihat perubahan code secara real-time tanpa perlu rebuild aplikasinya dari awal.
2. Walaupun sama-sama dari Firebase, perannya beda jauh:

- a. Firebase Authentication: Ini khusus ngurusin autentikasi. Perannya untuk memvalidasi identitas karyawan (login/register), nge-generate token sesi, dan memastikan siapa user yang sedang mengakses aplikasi.
 - b. Firebase Firestore: Ini NoSQL database-nya. Perannya untuk menyimpan data absensi yang terstruktur, seperti catatan waktu (timestamp), lokasi (latitude/longitude), dan ID karyawan tersebut.
3. Validasi radius 500 meter itu wajib biar kita yakin karyawan beneran ada di area pabrik pas absen. Lalu untuk Risiko Kalau fitur ini tidak diimplementasikan, maka akan menjadi sangat rentan terjadi time theft. Karyawan bisa titip absen atau clock in santai dari rumah. Risiko keamanannya juga mencakup penggunaan aplikasi Fake GPS (spoofing). Kalau validasinya nggak ketat, integritas seluruh data absensi bisa dibilang cacat.
4. Haversine adalah rumus matematika yang digunakan untuk menghitung jarak terpendek antara dua titik di atas permukaan bola (seperti bumi) berdasarkan *latitude* dan *longitude*. Dalam aplikasi, saat user menekan absen, kita ambil koordinat *device* dan koordinat pabrik, lalu dihitung menggunakan rumus ini:

$$d = 2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

Jika hasil d di bawah 500 meter maka proses dilanjutkan. Apabila lebih maka absen akan ditolak

5. - Geofencing: Ini memanfaatkan service dari OS (Android/iOS). Kita bisa membuat "pagar virtual", dan OS akan memberikan trigger event otomatis ketika device masuk atau keluar area tersebut, walaupun aplikasinya sedang berjalan di background.
 - Haversine: Ini murni distance calculation secara matematis. Proses perhitungannya cuma jalan ketika aplikasinya dibuka di foreground dan fungsinya dipanggil secara spesifik (misal, saat tombol ditekan).
6. - StatelessWidget: Komponen UI yang immutable atau statis. Contoh di aplikasi ini: Teks "Selamat Datang" atau logo perusahaan di halaman utama yang nggak perlu berubah bentuk.
 - StatefulWidget: Komponen UI yang state-nya bisa berubah dinamis karena interaksi user atau data baru. Contoh: Tombol "Clock In" yang berubah menjadi loading spinner saat ditekan sambil menunggu respons jaringan.
7. Kalau aplikasinya makin kompleks, state management itu esensial. dan apabila tidak ada state management maka akan menimbulkan masalah seperti berikut:
 Jika kita cuma pakai setState, kita akan mengalami prop drilling—ngelempar data (seperti status login atau lokasi user) dari root widget ke widget yang paling dalam lewat parameter constructor berkali-kali. Kodenya jadi kotor. Selain itu, memanggil setState sering me-rebuild keseluruhan pohon widget secara tidak efisien, padahal mungkin yang perlu di-update hanya satu teks kecil. Bikin performa aplikasi jadi berat.
8. Library ini menggunakan machine learning on-device untuk membaca kontur dan wajah langsung dari stream kamera tanpa harus mengirim data ke server.

- a. minFaceSize: Parameter ini menentukan rasio ukuran wajah minimal terhadap gambar. Kalau diset 0.15, library akan mengabaikan wajah-wajah kecil (misalnya orang yang lewat di background), dan fokus ke wajah yang paling dekat dengan kamera.
 - b. trackingEnabled: Kalau diaktifkan, algoritma akan memberikan ID unik untuk setiap wajah di berbagai frame. Berguna banget untuk liveness detection atau memastikan wajah yang di-scan tidak berganti orang di tengah proses.
9. Berikut adalah logic flow ketika tombol absen ditekan:



```
1  function onClockInPressed():
2      showLoadingState()
3
4      userLocation = getCurrentGPSLocation()
5      officeLocation = getOfficeLocation()
6
7      distance = calculateHaversine(userLocation, officeLocation)
8
9      if distance > 500:
10         hideLoadingState()
11         showNotificationError("Outside Area: Anda berada di luar radius pabrik.")
12         return
13
14     // Jika jarak valid, lanjut verifikasi wajah
15     proceedFaceVerification()
```

10. Data flow FCM untuk notifikasi shift bekerja seperti ini:
- a. Server (Backend): Kita jalankan CRON Job yang mengecek jadwal secara periodik. Saat waktu menunjukkan H-15 menit shift, server mengirimkan HTTP request ke API FCM Google dengan membawa payload notifikasi dan target Device Token user (atau Topic tertentu).
 - b. FCM Backend: Server Google memproses request tersebut dan mencari perangkat yang terdaftar dengan token itu.
 - c. Client Device: Jika HP terhubung ke internet, OS akan menerima payload tersebut dan langsung memunculkan push notification "Waktu absen tiba!" di layar HP, meskipun aplikasi sedang ditutup.
11. Konsepnya adalah memisahkan logic menjadi beberapa layer agar mudah di-maintain dan scalable.
- a. Presentation Layer: Mengurus UI dan interaksi user (Flutter Widgets, Provider/Riverpod).
 - b. Domain Layer: Berisi business logic dan entitas utama aplikasi (misal: use case perhitungan absen).
 - c. Data Layer: Tempat mengambil dan menyimpan data, seperti implementasi API atau Firebase Firestore.

- d. Dependency Rule: Alur dependency-nya mengarah ke dalam. Presentation dan Data boleh bergantung pada Domain, tapi Domain sama sekali nggak boleh tahu menahu urusan UI atau dari mana datanya berasal.
12. Agar user lain tidak bisa melihat data absen orang lain, maka diterapkan aturan seperti berikut:



```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     match /attendance/{docId} {
5       // 1. User cuma bisa membaca datanya sendiri
6       allow read: if request.auth != null && request.auth.uid == resource.data.userId;
7
8       // 2. User cuma bisa membuat absen atas namanya sendiri
9       allow create: if request.auth != null && request.auth.uid == request.resource.data.userId;
10
11      // 3. Mencegah manipulasi data absen yang sudah masuk (immutable)
12      allow update, delete: if false;
13    }
14  }
15 }
```

13. - Risiko: Karyawan bisa merasa pergerakannya dimata-matai di luar jam kerja. Selain itu, ada kekhawatiran data selfie biometrik mereka bocor dan disalahgunakan.
- Mitigasi Teknis: Mintalah perizinan lokasi dengan tipe "While Using The App", bukan "Always". Aplikasi hanya boleh menarik kordinat sesaat saat tombol ditekan. Untuk wajah, pastikan ML Kit berjalan sepenuhnya on-device, sehingga foto tidak perlu dikirim ke server. Jika foto wajib diunggah untuk bukti, batasi akses bucket Storage-nya dengan rules yang ketat.
14. - Firebase Storage: Memang didesain secara spesifik untuk menyimpan blob atau file biner berukuran besar (seperti gambar JPEG/PNG atau video).
- Firebase Firestore: Didesain untuk data tekstual yang ringan dan terstruktur. Kalau kita memaksa mengubah selfie jadi Base64 String lalu menyimpannya di Firestore, selain bisa menabrak limit ukuran dokumen (1MB/dokumen), read/write operations-nya akan sangat mahal dan membuat aplikasi berjalan lambat. Lebih baik simpan fotonya di Storage, dan simpan URL download-nya saja di Firestore.
15. Analisis design dan fitur izin/cuti:
- a. UI/UX Flow:
 - i. User membuka menu "Pengajuan Izin" lalu Mengisi form (Pilih tipe, tanggal mulai/selesai, alasan, upload bukti surat dokter) lalu Submit. Di layar user statusnya "Pending".
 - ii. Admin membuka dashboard lalu Memeriksa detail request dan bukti attachment lalu Menekan "Approve/Reject" lalu Status di aplikasi user update secara real-time.
 - b. Data Model Firestore (leave_requests):
 - i. id: String

- ii. `userId`: String
- iii. `startDate`: Timestamp
- iv. `endDate`: Timestamp
- v. `reason`: String
- vi. `status`: String (contoh: "pending", "approved", "rejected")
- vii. `attachmentUrl`: String (URL dari Firebase Storage)
- viii. `createdAt`: Timestamp
- c. Notifikasi ke Admin:

Kita bisa menggunakan Firebase Cloud Functions. Saat ada trigger dokumen baru terbuat (on-create) di collection `leave_requests`, Cloud Function akan mengeksekusi skrip untuk mengirim FCM push notification langsung ke Topic `admin_channel`, agar tim HR mendapat peringatan seketika di device mereka.

BAGIAN 3: MUHAMMAD RADHITYA NAUFAL ()

BAGIAN 4: IBNU FAJAR (103012300122)

1. Flutter dipilih untuk aplikasi Factory Attendance karena lebih praktis dibanding Kotlin. Kalau pakai Kotlin, aplikasi cuma bisa dipakai di Android, sedangkan untuk iPhone harus bikin ulang pakai Swift. Nah, Flutter punya konsep single codebase, jadi cukup satu kali coding dengan Dart lalu bisa langsung dijalankan di Android dan iOS. Selain itu, tampilan UI di Flutter lebih konsisten di berbagai ukuran HP, jadi desain aplikasi tetap rapi. Flutter juga punya fitur hot reload, jadi saat coding atau cari bug, perubahan bisa langsung muncul tanpa build ulang aplikasi, sehingga development jadi lebih cepat.
2. Firebase Authentication dan Firebase Firestore itu sebenarnya beda fungsi. Kalau Firebase Authentication fokusnya buat login dan identitas user, sedangkan Firebase Firestore fokusnya buat menyimpan data aplikasi.

Di aplikasi Factory Attendance, Firebase Authentication dipakai buat proses login akun karyawan, misalnya login pakai email dan password. Jadi sistem bisa tahu siapa user yang masuk dan memastikan cuma user yang punya akun yang bisa akses aplikasi atau melakukan absensi. Sedangkan Firebase Firestore dipakai buat nyimpen data-data yang ada di aplikasi.

Contoh seperti data profil karyawan, jam check in dan check out, status kehadiran, sampai lokasi absensi user. Jadi setelah user berhasil login lewat Authentication, data absensinya nanti disimpan ke Firestore.

poinnya tu Authentication itu buat tau siapa usernya, sedangkan Firestore itu buat nyimpen data yang dipakai aplikasi.

3. GPS validation dengan radius 500 meter diperlukan supaya aplikasi bisa memastikan user benar-benar ada di sekitar area kantor atau pabrik saat melakukan absensi. Jadi aplikasi bakal ngecek jarak antara lokasi user dan lokasi kantor, kalau masih dalam radius yang ditentukan maka absensi bisa dilakukan. Radius 500 meter juga dipakai karena GPS kadang tidak selalu akurat, jadi ada sedikit toleransi lokasi.

Kalau fitur ini tidak diterapkan, risikonya besar karena user bisa melakukan absensi dari mana aja, misalnya dari rumah atau tempat lain tanpa benar-benar datang ke kantor. Selain itu, bisa terjadi titip absen atau manipulasi kehadiran, sehingga data absensi jadi kurang valid dan keamanan sistem attendance jadi berkurang.

contohnya ada salah satu teman saya yang melakukan absensi namun orangnya sedang di luar bandung.

4. Haversine Formula itu bisa dibilang rumus buat ngitung jarak antara dua lokasi dari titik GPS. Jadi rumus ini pakai data latitude dan longitude buat cari tau seberapa jauh jarak satu tempat ke tempat lain.

Di aplikasi Factory Attendance, rumus ini dipakai buat ngitung jarak antara lokasi user sama lokasi kantor. Misalnya pas user mau absen, aplikasi ambil lokasi GPS user lalu dibandingin sama lokasi kantor. Kalau jaraknya masih dekat atau masuk radius yang ditentukan, user bisa check in. Tapi kalau terlalu jauh, absensi biasanya ga bisa dilakukan.

Jadi Haversine Formula dipakai supaya aplikasi bisa ngecek apakah user beneran dekat kantor atau ngga saat mau absen.

5. Geofencing itu bisa dibilang fitur buat bikin batas area virtual di sekitar lokasi tertentu, misalnya kantor atau pabrik. Jadi di aplikasi Factory Attendance, sistem bikin area absensi di sekitar kantor, lalu aplikasi bakal ngecek apakah user masih ada di dalam area itu atau sudah keluar. Kalau user masih masuk area, biasanya absensi bisa dilakukan.

Bedanya sama simple distance calculation (Haversine), kalau Haversine lebih fokus buat ngitung jarak antara lokasi user dan kantor, misalnya hasilnya 200 meter atau 700 meter. Sedangkan geofencing lebih ke menentukan user ada di dalam area atau di luar area yang sudah dibuat.

Jadi Haversine itu ngitung jarak, sedangkan geofencing itu bikin batas area buat ngecek user masih masuk wilayah absensi atau ngga.

6. Perbedaan StatefulWidget dan StatelessWidget di Flutter itu ada di perubahan tampilannya saat aplikasi dipakai.

StatelessWidget dipakai kalau tampilannya tetap dan biasanya ngga berubah-ubah. Jadi widget ini cuma buat nampilin informasi aja. Contohnya di aplikasi Factory

Attendance seperti logo aplikasi, judul “Factory Attendance”, atau halaman informasi, karena isinya cuma ditampilkan tanpa ada perubahan.

Sedangkan StatefulWidget dipakai kalau tampilannya bisa berubah tergantung kondisi atau aksi user. Contohnya di Factory Attendance seperti tombol check in/check out atau status absensi. Misalnya awalnya tombol bertuliskan “Check In”, lalu setelah ditekan berubah jadi “Sudah Absen”, berarti widget itu lebih cocok pakai StatefulWidget karena ada perubahan data atau tampilan.

Jadi, StatelessWidget dipakai buat tampilan yang tetap, sedangkan StatefulWidget dipakai kalau ada perubahan tampilan saat aplikasi berjalan.

7. State management seperti Provider atau Riverpod diperlukan di aplikasi Factory Attendance supaya data di aplikasi bisa diatur dengan lebih rapi dan gampang dipakai di banyak halaman. Misalnya data user login, status absensi, atau data lokasi user, jadi bisa langsung dipakai tanpa harus kirim data terus-menerus antar halaman.

Kalau cuma mengandalkan setState, biasanya masih aman buat aplikasi kecil, tapi makin ribet kalau aplikasinya mulai banyak fitur. Misalnya data user login harus dipakai di halaman dashboard, profil, dan attendance, jadi kita harus kirim data berkali-kali atau update manual di banyak tempat.

Selain itu, setState cuma update tampilan di widget tertentu aja. Jadi kadang data bisa ngga sinkron antar halaman atau UI jadi susah diatur. Misalnya status absensi user sudah berubah, tapi ada halaman lain yang belum ikut update. Karena itu state management dipakai supaya data lebih teratur, gampang diakses, dan perubahan data bisa langsung kebaca di bagian aplikasi yang butuh.

8. ML Kit Face Detection itu library yang dipakai buat ngecek ada wajah atau ngga di kamera atau foto. Di aplikasi Factory Attendance, fitur ini dipakai pas selfie absensi biar sistem bisa memastikan user beneran foto wajah saat check in atau check out.

Cara kerjanya kurang lebih, kamera ngambil gambar lalu ML Kit bakal mendeteksi bentuk wajah dari foto itu, misalnya area mata, hidung, mulut, atau posisi kepala. Dari situ sistem bisa tau apakah ada wajah yang terdeteksi atau ngga. Kalau ada wajah, absensi bisa lanjut, kalau ngga biasanya disuruh foto ulang.

parameter yang bisa diatur misalnya:

- minfacesize : buat ngatur ukuran minimal wajah yang bisa dibaca. Jadi kalau wajah terlalu kecil atau terlalu jauh dari kamera, bisa aja ngga ke-detect.
- trackingenabled : buat melacak wajah saat kamera aktif, jadi kalau wajah bergerak masih bisa tetap kebaca.
- performancemode : buat ngatur apakah sistem mau lebih fokus ke kecepatan atau hasil deteksi yang lebih detail.

- landmarkmode : buat bantu mendeteksi bagian wajah seperti mata, hidung, atau mulut.

Jadi, ML Kit Face Detection ini dipakai supaya aplikasi bisa ngecek kalau yang selfie itu memang ada wajah, jadi proses absensi lebih valid dan ngga asal upload foto.

9. Kalau user menekan tombol “Clock In” saat berada di luar radius kantor

BEGIN

WHEN button "Clock In" pressed

GET current user location

IF location == null THEN

SHOW "Location not found"

STOP

END IF

SET officeLatitude

SET officeLongitude

CALCULATE distance using Haversine Formula

IF distance <= 500 meter THEN

SAVE attendance data

SHOW "Clock In Success"

ELSE

SHOW "Outside Area"

END IF

END

Jadi alurnya, setelah tombol ditekan aplikasi bakal ambil lokasi user dulu, lalu hitung jaraknya ke kantor. Kalau masih masuk radius absensi proses lanjut, tapi kalau terlalu jauh langsung muncul error “**Outside Area**” dan absensi ngga diproses.

10. Firebase Cloud Messaging (FCM) dipakai buat ngirim push notification dari server ke HP user. Di aplikasi Factory Attendance, misalnya dipakai buat ngingetin user 15 menit sebelum jam shift dimulai supaya jangan lupa absen.

Cara kerjanya kurang lebih, server atau backend bakal ngecek jadwal shift user. Kalau waktunya sudah mendekati (misalnya H-15 menit), server akan mengirim request ke FCM. Setelah itu FCM jadi perantara yang meneruskan notifikasi ke device user berdasarkan token device yang sudah tersimpan sebelumnya. Nanti notifikasi muncul di HP, misalnya "Shift kamu mulai 15 menit lagi, jangan lupa Clock In".

Arsitektur data flow:

Database Shift User

Server / Backend cek jadwal shift

Waktu shift -15 menit ?

Kirim request notification ke FCM

Firebase Cloud Messaging (FCM)

FCM kirim notification ke device token user

HP User menerima push notification

Notifikasi muncul di layar

"15 menit lagi waktu Clock In"

Jadi, server yang menentukan kapan notif dikirim, sedangkan FCM jadi penghubung yang membantu mengirim notifikasi sampai ke HP user.

11. Clean Architecture itu konsep buat bikin susunan project lebih rapi supaya kode aplikasi ngga gampang berantakan, apalagi kalau fiturnya mulai banyak. Jadi di aplikasi Factory Attendance, tiap bagian punya tugas masing-masing dan ngga campur aduk. Misalnya ada bagian yang fokus buat tampilan aplikasi, ada yang ngurus logic, dan ada yang bagian ambil atau simpan data.

Bagian pertama biasanya Presentation Layer, yaitu bagian yang langsung dilihat user, seperti halaman login, tombol clock in, halaman attendance, atau status absensi. Bagian ini fokusnya cuma nampilin data dan nerima input dari user. Setelah itu ada Domain Layer, yaitu tempat logic utama aplikasi berjalan. Contohnya kayak ngecek apakah user masih masuk radius kantor, apakah user sudah absen, atau aturan attendance lainnya. Terakhir ada Data Layer, yang tugasnya ambil atau simpan data, misalnya login lewat Firebase Authentication, nyimpen data absensi ke Firestore, atau ambil lokasi GPS user.

Di Clean Architecture juga ada aturan alur kerja atau dependency rule, jadi prosesnya berjalan dari Presentation > Domain > Data. Jadi misalnya user pencet tombol clock in, tampilannya bakal kirim request ke logic dulu, baru logic itu minta data ke

Firebase atau GPS. Dengan cara ini kode jadi lebih rapi, gampang dicari kalau ada bug, dan kalau ada perubahan fitur biasanya nggak bikin bagian lain ikut berantakan.

12. Buat ngamanin data attendance di firebase firestore, biasanya dipakai firestore security rules supaya user cuma bisa akses data miliknya sendiri dan nggak sembarang liat atau ubah data orang lain. di aplikasi factory attendance ini penting banget, karena data absensi tiap user sifatnya pribadi.

3 security user:

- authenticated user (user harus login terlebih dahulu) : jadi cuma user yang sudah login yang boleh akses data attendance. kalo belum login otomatis ditolak
- user cuma bisa liat data miliknya sendiri : misalnya setiap data attendance punya user id, lalu sistem ngecek apakah user id di database sama dengan id user yang login. kalau beda, akses ditolak, jadi user A nggak bisa buka attendance user B.
- user cuma bisa edit atau tambah data miliknya sendiri: saat melakukan clock in atau clock out, user cuma boleh membuat atau update data attendance yang sesuai akun dia, bukan data orang lain.

jadi firestore security rules dipakai buat membatasi akses data berdasarkan akun yang sudah login supaya data attendance tetap aman dan user cuma bisa liat data miliknya sendiri.

13. Dalam aplikasi Factory Attendance, pengumpulan data GPS dan foto selfie memang membantu validasi absensi, tapi tetap ada risiko privasi yang perlu diperhatikan. Misalnya dari data GPS, aplikasi bisa tau lokasi user saat absen, sehingga kalau data bocor atau disalahgunakan, lokasi user bisa diketahui orang lain. Sedangkan dari foto selfie, ada risiko penyalahgunaan identitas atau foto dipakai tanpa izin kalau penyimpanannya kurang aman.

Biar risiko ini lebih kecil, dari sisi teknis aplikasi bisa melakukan beberapa hal. Pertama, membatasi akses data dengan Firebase Security Rules, jadi user cuma bisa akses data miliknya sendiri dan admin tertentu saja yang punya izin lebih. Kedua, menyimpan data secara aman, misalnya foto selfie atau data attendance disimpan di database dengan akses terbatas dan menggunakan koneksi terenkripsi supaya data nggak gampang diambil pihak lain. Ketiga, mengambil data seperlunya saja, misalnya GPS hanya diambil saat proses clock in/clock out, bukan dilacak terus selama aplikasi berjalan. Jadi aplikasi cuma ambil data yang memang dibutuhkan buat validasi absensi.

Selain itu, aplikasi juga bisa menerapkan autentikasi user supaya cuma akun yang valid yang bisa akses data attendance, dan membatasi siapa saja yang bisa ngeliat data

selfie atau lokasi user. Dengan begitu, risiko privasi dari GPS dan selfie bisa lebih diminimalisir dan data user tetap lebih aman.

14. Firebase Storage dan Firebase Firestore itu beda fungsi walaupun sama-sama tempat nyimpen data. Kalau Firestore lebih cocok buat nyimpen data ringan atau informasi berbentuk teks, misalnya nama user, jam clock in, status kehadiran, atau link foto. Sedangkan Firebase Storage lebih cocok buat nyimpen file berukuran lebih besar seperti foto, video, atau dokumen.

Di aplikasi Factory Attendance, foto selfie hasil absensi biasanya disimpan di Firebase Storage karena bentuknya file gambar. Setelah foto berhasil di-upload, biasanya yang disimpan ke Firestore cuma link atau path fotonya beserta data attendance lain, misalnya jam masuk dan lokasi user.

Alasannya foto disimpan di Storage, bukan Firestore karena lebih efisien dan memang dibuat khusus buat file. Kalau foto dipaksa disimpan langsung di Firestore, penyimpanan bisa lebih berat, akses jadi kurang optimal, dan biaya penyimpanan juga bisa lebih besar. Jadi pembagiannya biasanya Firestore buat data attendance, sedangkan Storage buat file selfie absensinya.

15. kalau ditambah fitur izin/cuti, alurnya bisa dibuat simpel biar gampang dipakai user dan admin.

a. ui/ux flow request sampai approval

user masuk ke menu izin/cuti, lalu pilih jenis izin (izin atau cuti), isi alasan, pilih tanggal mulai dan selesai, lalu bisa tambah bukti kalau diperlukan (misalnya surat dokter). setelah itu user tekan tombol submit request. status awal otomatis jadi pending.

setelah request masuk, admin bakal dapet data pengajuan di halaman approval. admin bisa liat detail pengajuan seperti nama user, alasan, tanggal izin, dan bukti pendukung. setelah dicek, admin tinggal pilih approve atau reject. hasil keputusan nanti muncul di aplikasi user lewat status request atau notifikasi.

flow sederhananya:

user buka menu izin/cuti

isi form izin/cuti

submit request

data masuk ke firestore (status: pending)

admin liat request baru

approve / reject

status user diperbarui + notifikasi hasil

b. data model firestore buat izin/cuti

buat nyimpen data izin atau cuti, di firestore bisa dibuat collection misalnya `leave_requests`. di dalamnya berisi data penting yang dibutuhkan selama proses pengajuan sampai approval. misalnya ada `userId` buat nandain siapa yang mengajukan, `namaUser` buat nampilin nama user, `jenisIzin` apakah izin biasa atau cuti, lalu alasan buat menjelaskan kenapa user mengajukan izin.

selain itu ada `tanggalMulai` dan `tanggalSelesai` supaya admin bisa tau rentang waktu izin atau cutinya. kalau dibutuhkan bukti seperti surat dokter, bisa ditambah `buktiFile` yang isinya link file dari storage. lalu ada status yang isinya `pending`, `approved`, atau `rejected` buat nandain apakah pengajuan masih diproses, diterima, atau ditolak. terakhir ada `createdAt` buat nyimpen kapan request itu dibuat supaya data lebih rapi dan gampang dicek.

c. notifikasi ke admin saat ada request baru

pas user submit izin/cuti, sistem bisa langsung kirim push notification pakai FCM ke admin. misalnya notif seperti: “ada request izin baru dari ibnu fajar, silakan cek approval.”

jadi admin bisa langsung tau ada pengajuan baru tanpa harus buka aplikasi terus-terusan.